

CHAPTER 3: METHODOLOGY

3.1 Research Approach and Design Science Paradigm

This study adopts the **Design Science Research (DSR)** paradigm in Computer Science to address the problem of transaction-splitting (smurfing) and threshold evasion in banking networks. DSR focuses on the creation and evaluation of innovative IT artifacts (such as models, methods, and systems) to solve identified organizational problems.

In this context, the research artifact is **Fraudstruct**—a hybrid real-time streaming feature store and graph-adversarial machine learning library. The methodology is structured into five iterative phases:

1. **Problem Identification:** Analyzing the limitations of static rule-based threshold engines in Nigerian commercial banking.
 2. **Objectives of the Solution:** Designing a low-latency pipeline capable of GNN-level topological modeling within a $< 50\text{ms}$ transaction execution budget.
 3. **Artifact Design and Development:** Engineering a decoupled three-tier architecture utilizing Simplifying Graph Convolutions (SGC) and sliding dequeues.
 4. **Demonstration:** Implementing a simulated banking stream containing organic transactions and coordinated multi-hop smurfing attacks.
 5. **Evaluation:** Benchmarking decision accuracy, recall, and transaction processing latencies against standard banking SLAs.
-

3.2 Graph Formulation and Topological Modeling

Financial transaction systems are modeled as a dynamic, directed temporal graph.

$$G(t) = (V, E(t))$$

Where:

- $V = \{v_1, v_2, \dots, v_n\}$ represents the set of nodes (accounts, devices, or merchant terminals).
- $E(t) = \{e_1, e_2, \dots, e_m\}$ represents the set of edges (transactions).

Each transaction $e_i \in E(t)$ is modeled as a directed edge:

$$e_i = (u, v, a, t, c)$$

where u is the source account, v is the destination account, $a \in \mathbb{R}^+$ is the transaction amount, t is the transaction timestamp, and c is the transaction channel.

Node Feature Extraction

At any time t , each node $v \in V$ is mapped to a feature vector $x_v(t) \in \mathbb{R}^d$. The baseline node features represent local transactional velocity and topological status:

1. **In-Degree ($d_{in}(v)$):** The number of incoming transactions to account v within the active network.
2. **Out-Degree ($d_{out}(v)$):** The number of outgoing transactions from account v .

3. **Inbound Sum** ($S_{in}(v)$): The cumulative volume of funds received:

$$S_{in}(v) = \sum_{(u,v,a,t',c) \in E(t)} a$$

4. **Outbound Sum** ($S_{out}(v)$): The cumulative volume of funds sent:

$$S_{out}(v) = \sum_{(v,w,a,t',c) \in E(t)} a$$

5. **PageRank Score** ($PR(v)$): Measures structural importance within the transaction network:

$$PR(v) = \frac{1-d}{N} + d \sum_{u \in B_v} \frac{PR(u)}{d_{out}(u)}$$

where B_v is the set of all nodes directing transactions to v , $d_{out}(u)$ is the out-degree of node u , and d is the damping factor (set to 0.85).

3.3 Mathematical Modeling of Adversarial Smurfing (Transaction Splitting)

Fraudsters coordinate campaigns by dividing a large sum S across multiple intermediate proxy nodes (mules) to bypass velocity rules. We formalize this coordinated attack as a **Constrained Flow Optimization Problem**.

Let:

- s be the source account (compromised account containing funds).
- d be the target destination account (cash-out node).
- $M = \{m_1, m_2, \dots, m_k\}$ be the set of intermediary mule accounts.
- θ_{amount} be the transaction alert threshold (e.g., 5,000,000 NGN).
- θ_{count} be the transaction count limit per rolling window W .

The adversary solves the following constraints:

1. Flow Conservation

The total sum routed out of s must equal the sum received by d , adjusted by a transaction fee decay factor $\gamma \in [0.95, 0.99]$:

$$\sum_{i=1}^k a(s, m_i) = S \quad \text{and} \quad \sum_{i=1}^k a(m_i, d) = \gamma \cdot S$$

2. Edge Evasion

No individual transaction between the source and a mule, or a mule and the destination, can exceed the alert threshold:

$$\forall i \in \{1, \dots, k\} : \quad a(s, m_i) < \theta_{amount} \quad \text{and} \quad a(m_i, d) < \theta_{amount}$$

3. Temporal Dispersion

To prevent triggering temporal velocity checks (e.g., multiple rapid transfers), the adversary space out transactions by introducing a temporal delta Δt :

$$t(s, m_{i+1}) - t(s, m_i) \geq \Delta t$$

By modeling this constraint space, Fraudstruct can generate realistic, mathematically valid synthetic attacks to test GNN classification performance.

3.4 Simplifying Graph Convolution (SGC) Mathematical Mechanics

Traditional Graph Convolutional Networks (GCNs) aggregate representations recursively using non-linear activation functions (like ReLU) at each layer:

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)} \right)$$

This recurrence introduces high computational complexity and latency because features must be propagated and transformed at query time. SGC resolves this by removing non-linearities and collapsing the weight matrices across layers.

Let S_{adj} be the normalized adjacency operator with self-loops:

$$S_{adj} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$$

For a K -layer GCN, SGC simplifies the propagation step to:

$$\bar{X} = S_{adj}^K X$$

This collapses the convolution into a **single matrix multiplication** which can be pre-computed in the background.

For node classification (mule vs. clean account), the target probability is computed via a sigmoid function:

$$P(y_i = 1 \mid \bar{x}_i) = \sigma(w^T \bar{x}_i + b) = \frac{1}{1 + e^{-(w^T \bar{x}_i + b)}}$$

The weights w and bias b are optimized via L2-regularized binary cross-entropy:

$$\mathcal{L}(w, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\sigma(w^T \bar{x}_i + b)) + (1 - y_i) \log(1 - \sigma(w^T \bar{x}_i + b))] + \lambda \|w\|_2^2$$

The parameter optimization is solved iteratively using gradient descent:

$$w \leftarrow w - \alpha \frac{\partial \mathcal{L}}{\partial w} \quad \text{and} \quad b \leftarrow b - \alpha \frac{\partial \mathcal{L}}{\partial b}$$

where α is the learning rate.

3.5 Methodological and Engineering Trade-offs

During the design and implementation of Fraudstruct, several key engineering trade-offs were evaluated:

1. Graph Representation: MultiDiGraph vs. Simple Directed Graph

- **MultiDiGraph:** Allows multiple directed edges between the same two accounts. This accurately reflects real banking behavior where Account A transfers money to Account B multiple times.
- **Simple Directed Graph:** Aggregates multiple transactions into a single weighted edge.
- **Decision:** We selected a **MultiDiGraph** for the Warm-Path memory model to ensure no transaction history is lost. However, for GNN feature propagation, we convert the multi-graph to a weighted **DiGraph** (where edge weight = transaction count) to prevent matrix dimension instability during degree matrix calculations.

2. Algorithmic Backend: PyTorch Geometric (PyG) vs. Pure NumPy/SciPy

- **PyTorch/PyG:** Provides GPU-accelerated deep graph neural networks.
- **Pure NumPy/SciPy:** Performs matrix convolutions in RAM using CPU vectorized arithmetic.
- **Decision:** We selected a **Pure NumPy** implementation for the runtime model. While PyTorch is highly efficient for model training, the installation of PyTorch and compiled C++ dependencies (such as PyG) in banking mainframes is frequently blocked by security policies. A NumPy-based SGC engine guarantees zero dependencies, fast CPU execution (sub-1ms for moderate subgraphs), and absolute environment portability.

3. Anomaly Scoring Model: Deep Autoencoders vs. SGC Node Classification

- **Deep Autoencoders:** Good at capturing reconstruction errors on unlabeled data.
- **SGC Node Classification:** Supervised/Semi-supervised neighborhood classification.
- **Decision:** Selected SGC. Smurfing is a coordinated behavioral pattern; simple reconstruction error checks frequently miss split pathways if the individual transactions look normal. Neighborhood aggregation is essential to propagate the malicious labels of known flagged accounts to adjacent mule nodes.

3.6 Experimental Setup and Verification Procedures

To validate the system's ability to detect smurfing under realistic banking operational constraints, we designed a simulated execution framework:

1. Data Generation

- **Organic Traffic:** Generates a network of 50 accounts transacting via exponential distributions to represent normal inter-account transfers.
- **Adversarial Injection:** Executes the Constrained Flow Optimization attack model. An attacker transfers 600,000 NGN to a beneficiary account split across 6 mule nodes, violating temporal spacing.

2. Evaluation Metrics

The artifact is evaluated against two groups of metrics:

- **Detection Efficacy:**
 - **Recall:** The fraction of attack transactions successfully flagged or blocked.
 - **Precision:** The ratio of true fraud flags to total triggered flags.
 - **GNN Generalization:** The anomaly score assigned to the attacker account after observing the mule graph topology.

- **Inference Efficiency:**

- **Mean Latency (ms):** The average time taken by the API to process a transaction request.
- **95th Percentile Latency (ms):** The upper bound latency representing the worst-case scenario.
- **SLA Breaches (%):** The fraction of transactions exceeding the 50ms transaction authorization timeout.